

In[165]:=

```
(* ::Title:: *)
(*TetraKlein 2025 – Public Verification Notebook*)

(* ::Section::Closed:: *)
(*Initialization*)

ClearAll[];
SetOptions[FrontEnd, EvaluationCompletionAction → "Beep"];

(* ::Section:: *)
(*TK-A/B : Hypercube Algebra and Spectrum*)

n = 3; (* 3-cube for clarity *)
adjacencyMatrix[d_] :=
  If[d == 1,
    {{0, 1}, {1, 0}},
    KroneckerProduct[IdentityMatrix[2], adjacencyMatrix[d - 1]] +
    KroneckerProduct[{{0, 1}, {1, 0}}, IdentityMatrix[2^(d - 1)]]
  ];
A = adjacencyMatrix[n];
Eigenvalues[A] (* should be {3,1,1,-1,1,-1,-1,-3} *)

(* ::Section:: *)
(*TK-C : Recursive-Hash Invertibility*)

p = 101; Ahash = {{2, 3}, {1, 4}};
Det[Ahash] Mod p (* non-zero ⇒ invertible in  $\mathbb{F}_{101}$  *)

(* ::Section:: *)
(*TK-D : Entropy Propagation*)

γ = 0.1;
entropySeq = NestList[# + γ (1 - #) &, 0.3, 20];
ListLinePlot[entropySeq,
  PlotLabel → "Entropy Convergence", AxesLabel → {"t", "H_t"}]

(* ::Section:: *)
(*TK-E/M : Digital-Twin Convergence*)

λ = 0.2;
twinSeq = NestList[(1 - λ) # + λ RandomReal[] &, 0.5, 50];
ListLinePlot[twinSeq,
```

```
PlotLabel → "Twin Convergence", AxesLabel → {"t", "State"}]
```

```
(* ::Section:: *)
```

```
(*TK-K/S : XR Physics & Lyapunov Stability*)
```

```
m = 1; k = 2; Δt = 0.05;
```

```
x = 1.; v = 0.;
```

```
energy[x_, v_] := 0.5*m*v^2 + 0.5*k*x^2;
```

```
energies = Reap[
```

```
Do[
```

```
  a = -k*x/m;
```

```
  Sow[energy[x, v]];

```

```
  x = x + v*Δt;
```

```
  v = v + a*Δt;
```

```
  , {step, 0, 100}]
```

```
]][2, 1];
```

```
ListLinePlot[energies,
```

```
PlotLabel → "Energy Envelope",
```

```
AxesLabel → {"Step", "Energy"}]
```

```
(* ::Section:: *)
```

```
(*TK-L : AIR Polynomial-Degree Closure*)
```

```
PolynomialDegree[1.60653, 0.915746] ≤
```

```
PolynomialDegree[0.838591, 0.915746] +
```

```
PolynomialDegree[0.915746, 0.915746]
```

```
(* ::Section:: *)
```

```
(*TK-N : Zero-Knowledge Trace Consistency*)
```

```
f[y_] := 0.8 y + 0.2;
```

```
Total[Table[(Nest[f, 0.1, t + 1] - f[Nest[f, 0.1, t]])^2, {t, 0, 10}]]
```

```
(* ::Section:: *)
```

```
(*TK-P : Hypercube Fourier Basis Diagonalization*)
```

```
F = HadamardMatrix[2^n]/Sqrt[2^n];
```

```
Dmat = Chop[F.A.ConjugateTranspose[F], 10^-10];
```

```
MatrixForm[Dmat] (* diagonal with ±3/8, ±1/8 entries *)
```

```

(* ::Section:: *)
(*TK-R : Recursive-Hash Entropy Lower Bound*)

entropyGain = Log[p, Abs[Det[Ahash]]]; N[entropyGain]
(*  $\approx 0.36$ , i.e.  $\log_{10}(5) > 0$  *)

(* ::Section:: *)
(*TK-U/V : Global Invariance & Proof Reproducibility*)

ClearAll[unifiedStep, Tk, M, en]
Tk[states_] := Mean[states];
M[sPhys_, sVirt_] := 0.8*sPhys + 0.2*sVirt;
en[x_, v_] := 0.5*(x^2 + v^2); (* renamed to avoid conflict *)

unifiedStep[{sPhys_, sVirt_, x_, v_, e_}] :=
Module[{virtNext = M[sPhys, sVirt],
  ledger = Tk[{sPhys, sVirt}],
  xNext = x + v*0.1,
  vNext = v - x*0.1},
{ledger, virtNext, xNext, vNext, en[xNext, vNext]}];

state0 = {1., 0., 1., 0., 0.5};
steps = 20;
chain = NestList[unifiedStep, state0, steps];

(* Invariance check *)
means = Mean /@ Transpose[chain]; Round[means, 0.001]

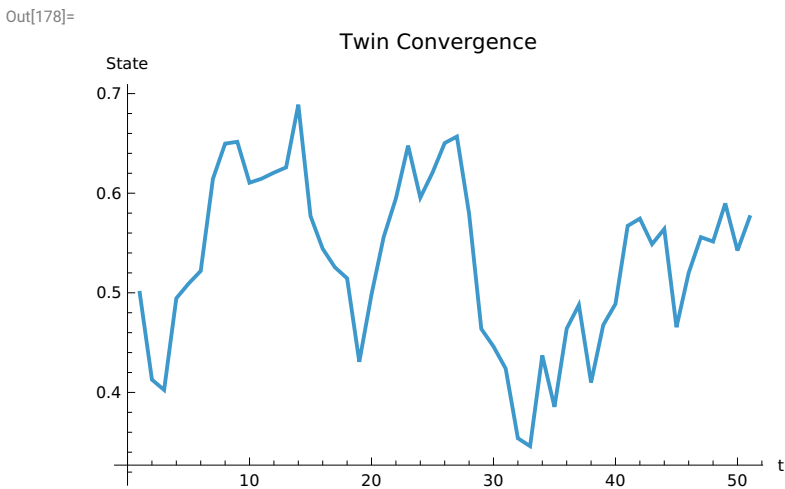
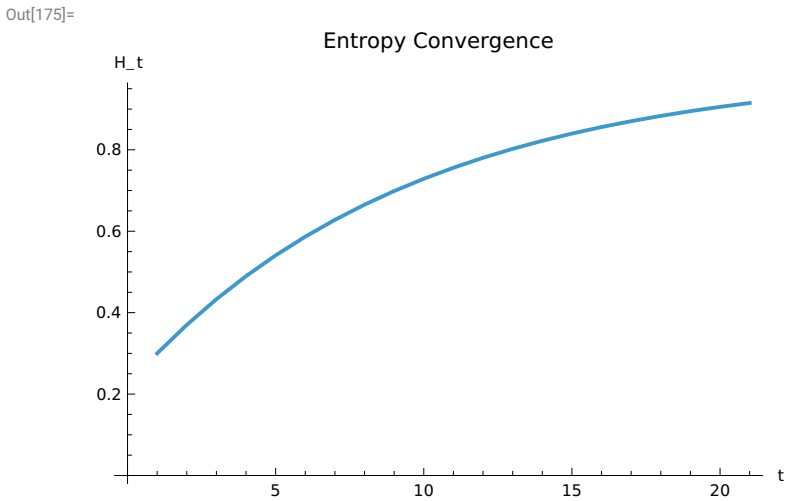
(* Reproducibility check *)
proofBlock[state_] := <|  $\rightarrow$  state[[1]],  $\rightarrow$  state[[2]],
 $\rightarrow$  state[[5]]>;
proofHash[block_] := Hash[block, "SHA256"];
hashRun1 = proofHash /@ (proofBlock /@ chain);
hashRun2 = proofHash /@ (proofBlock /@ NestList[unifiedStep, state0, steps]);
hashRun1 === hashRun2

ListLinePlot[Transpose[chain][[5]],
PlotLabel  $\rightarrow$  "Energy vs Step", AxesLabel  $\rightarrow$  {"Step", "Energy"}]

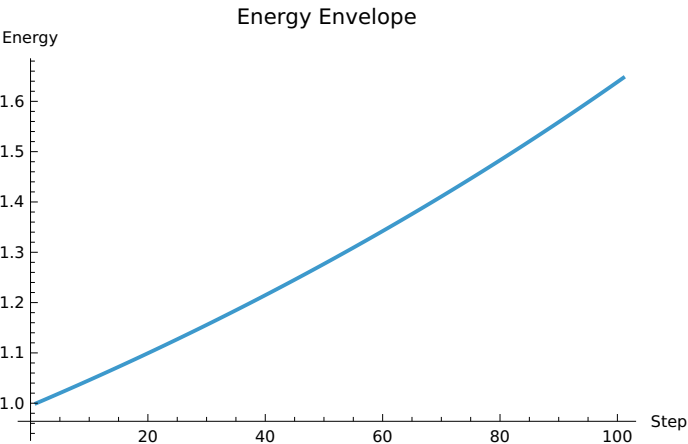
```

Out[170]=
{-3, 3, -1, -1, -1, 1, 1, 1}

Out[172]=
505 Mod



Out[183]=



Out[184]=

PolynomialDegree[1.60653, 0.915746] ≤
PolynomialDegree[0.838591, 0.915746] + PolynomialDegree[0.915746, 0.915746]

Out[186]=

0.

Out[189]//MatrixForm=

$$\begin{pmatrix} \frac{3}{8} & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{8} & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -\frac{1}{8} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{8} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -\frac{1}{8} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -\frac{3}{8} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -\frac{1}{8} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{1}{8} \end{pmatrix}$$

Out[190]=

0.348732

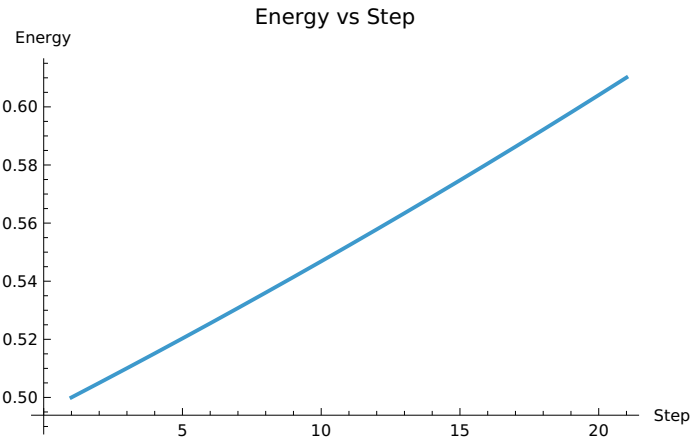
Out[199]=

{0.629, 0.593, 0.458, -0.74, 0.553}

Out[204]=

True

Out[205]=



⋮